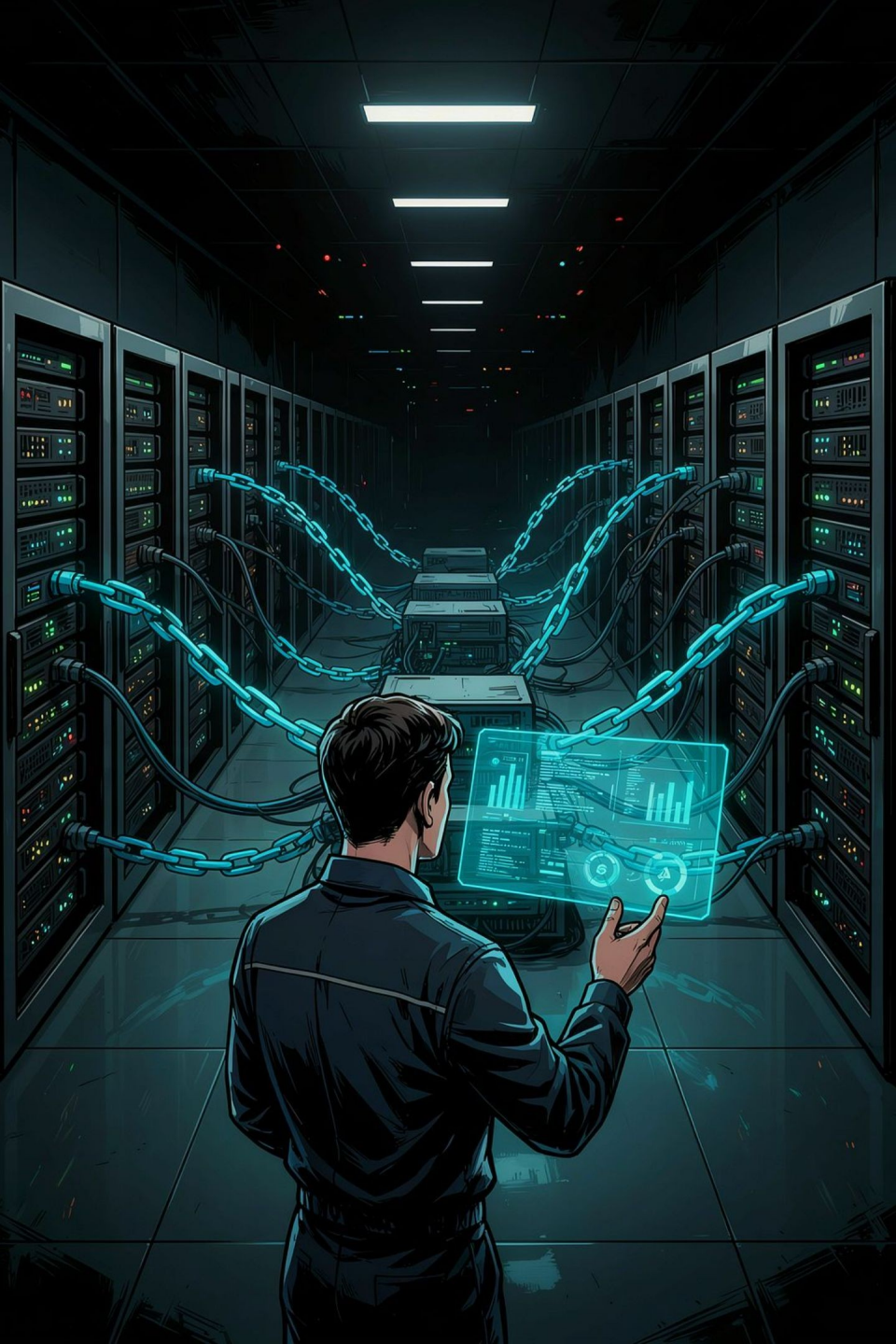




Module 11 – Supply Chain Security (Part 2)

Provenance • Runtime Security • Policy as Code

DEVSECOPS

KUBERNETES

ZERO TRUST

Supply Chain Security – Part 2

From Build Security to Runtime Trust

Topics Covered

01

Provenance

Cryptographic proof of software origin and build integrity

02

Runtime Security

Continuous workload monitoring and threat detection

03

Policy as Code

Automated governance and admission control

Learning Objectives

- Understand software provenance and attestation frameworks
- Secure workloads during execution with eBPF-based detection
- Automate governance with policy engines
- Prevent insecure deployments via admission controllers
- Build Zero Trust Kubernetes clusters end-to-end



Why Supply Chain Doesn't End After Signing

The Common Assumption



Many organizations treat image signing as the finish line — a dangerous misconception in modern threat environments.

Production Introduces New Risks

Compromised CI

Poisoned build systems inject malicious code

Stolen Keys

Signing keys exfiltrated and misused

Runtime Exploits

Attackers target running containers and kernel

Privilege Escalation

Misconfigurations enable unauthorized access

⚠ Supply chain security is a continuous lifecycle obligation — not a one-time gate.

What is Provenance?

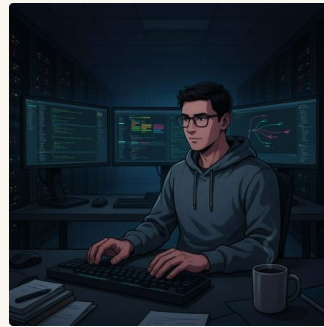
The Birth Certificate of Software

Software provenance answers a single critical question: "**Can we prove exactly where this software came from and how it was built?**" It creates a cryptographically verifiable record of every decision and input that produced a given artifact.



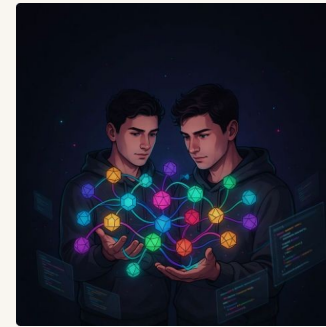
Source Identity

- Source repository URL
- Commit ID & branch
- Author identity



Build Context

- Build system & version
- Builder identity
- Build timestamp



Environment & Deps

- Dependencies resolved
- Compiler version
- Container base image

Why Provenance Matters

Without Provenance

Unknown Builder

No verified identity of who or what executed the build process

Unknown Source

Cannot confirm the artifact originated from trusted source code

Unknown Dependencies

Transitive dependencies may include malicious packages

No Forensic Trail

Incident investigation becomes extremely slow and unreliable

✓ With Provenance

Complete Audit Trail

Every build step is cryptographically recorded and immutable

Trusted Builder

Verified identity confirms authorized, unmodified build tooling

Verified Source

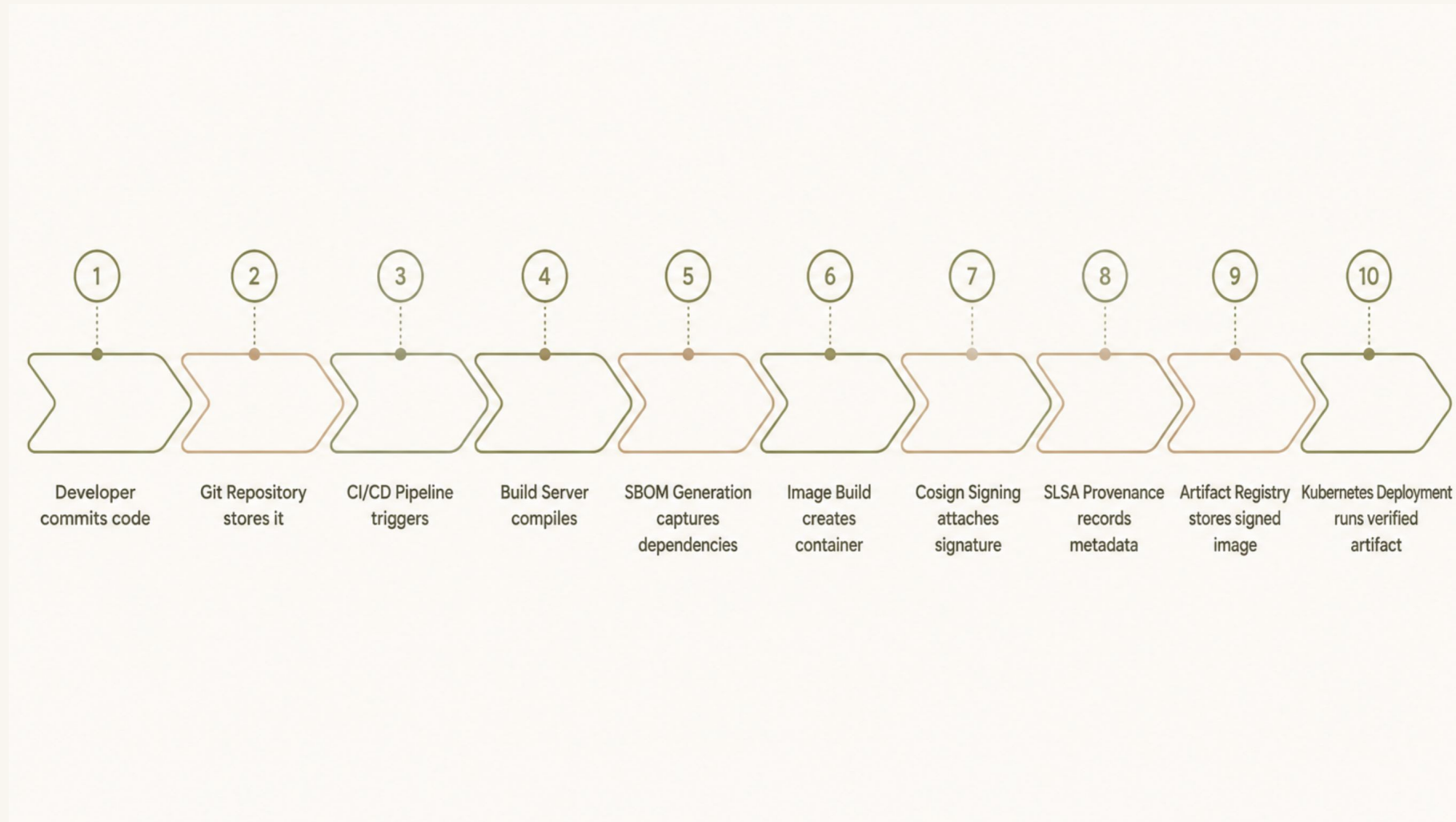
Commit-level traceability from artifact back to source code

Faster Response

Compliance-ready metadata accelerates incident investigation

Provenance Architecture

A production-grade provenance pipeline ensures that **every stage leaves cryptographic evidence** — from the developer's first commit to the final deployment in Kubernetes. No single step is trusted implicitly.



Understanding SLSA

Supply-chain Levels for Software Artifacts — Introduced by Google



Level 1 – Basic Provenance

Build process documented; provenance exists but is not authenticated



Level 2 – Hosted Build Service

Provenance generated and signed by a hosted build service (e.g., GitHub Actions)



Level 3 – Hardened Build Platform

Build platform hardened; source and build integrity strongly verified



Level 4 – Hermetic & Reproducible

Fully isolated, reproducible builds with no external network access



Higher SLSA level = lower supply chain risk. Enterprises should target SLSA Level 3 as their baseline.

Provenance Tools

The Enterprise Ecosystem for Build Integrity



Cosign

Signs container images and attestations using keyless or key-based cryptography. Integrates with OCI registries natively.



in-toto

Creates verifiable supply chain metadata, enforcing that each step in the pipeline was performed by an authorized party.



Tekton Chains

Automatically captures and signs build provenance within Tekton Pipelines, no manual steps required.



Syft

Generates Software Bill of Materials (SBOM) in SPDX and CycloneDX formats from container images and filesystems.



Rekor

Immutable transparency log for storing and querying provenance records. Part of the Sigstore ecosystem.



GitHub Attestations

Native artifact provenance built into GitHub Actions workflows, leveraging Sigstore for verification.

Runtime Security

Security doesn't stop after deployment.

Once a container is running, attackers shift their focus to exploitation at the OS and kernel level. Runtime security provides **continuous, real-time monitoring** of workload behavior — detecting anomalies that static scanning cannot catch.

- ❑ Runtime security continuously monitors workloads — shifting security left is necessary but not sufficient.

Active Attack Surfaces in Production

Containers

Running processes, filesystems, and spawned binaries

Kubernetes API

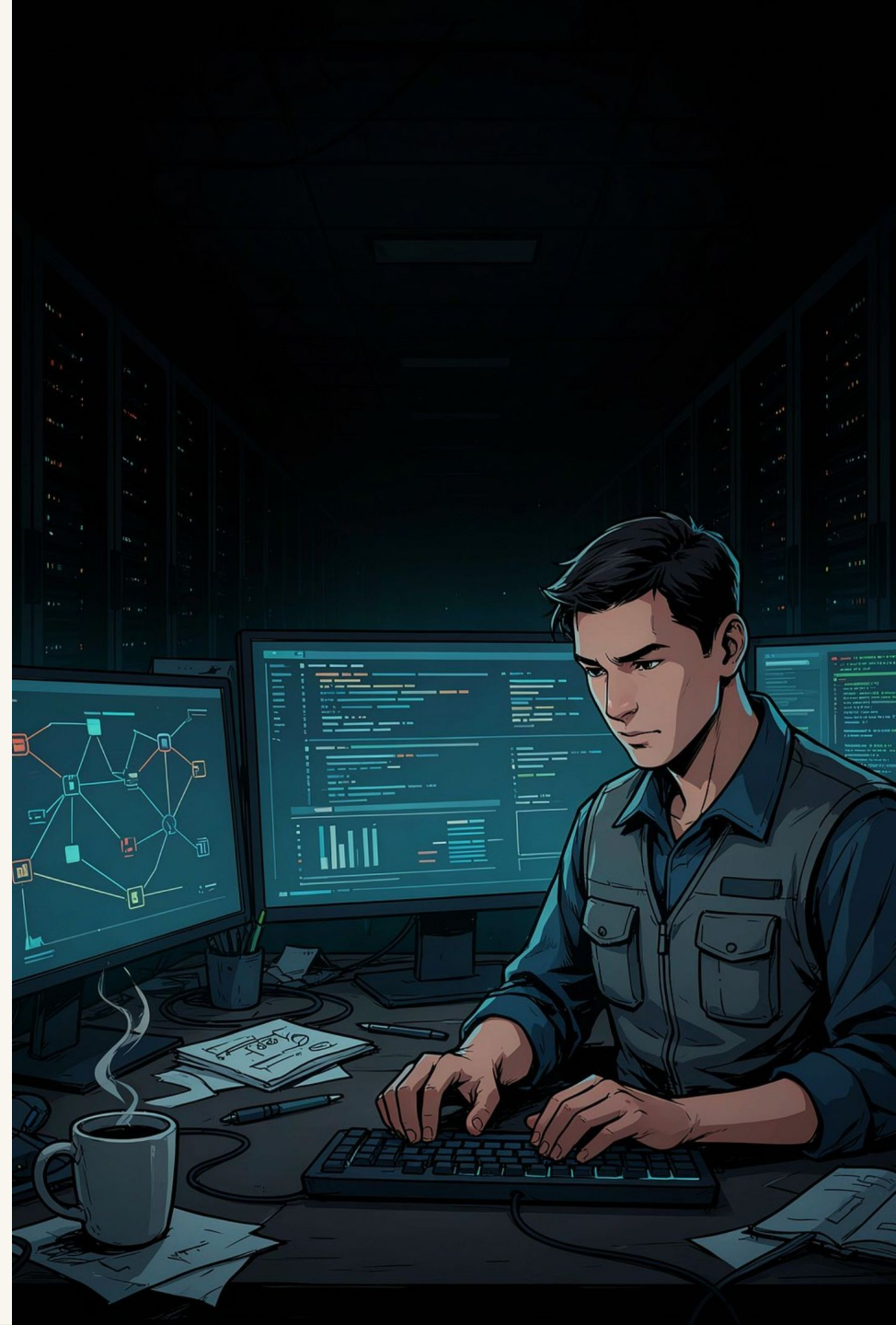
Unauthorized API calls and RBAC abuse

Linux Kernel

Syscall abuse and kernel exploits via container escape

Secrets & SAs

Service account token theft and secret exfiltration



Runtime Threat Landscape

Common Attacks Targeting Running Containers & Kubernetes



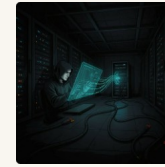
Reverse Shell & Container Escape

Attackers spawn unauthorized shells inside containers or exploit kernel vulnerabilities to break out of container isolation entirely.



Cryptojacking & Privilege Escalation

Malicious processes silently consume compute resources, while attackers exploit misconfigurations to gain root or cluster-admin privileges.



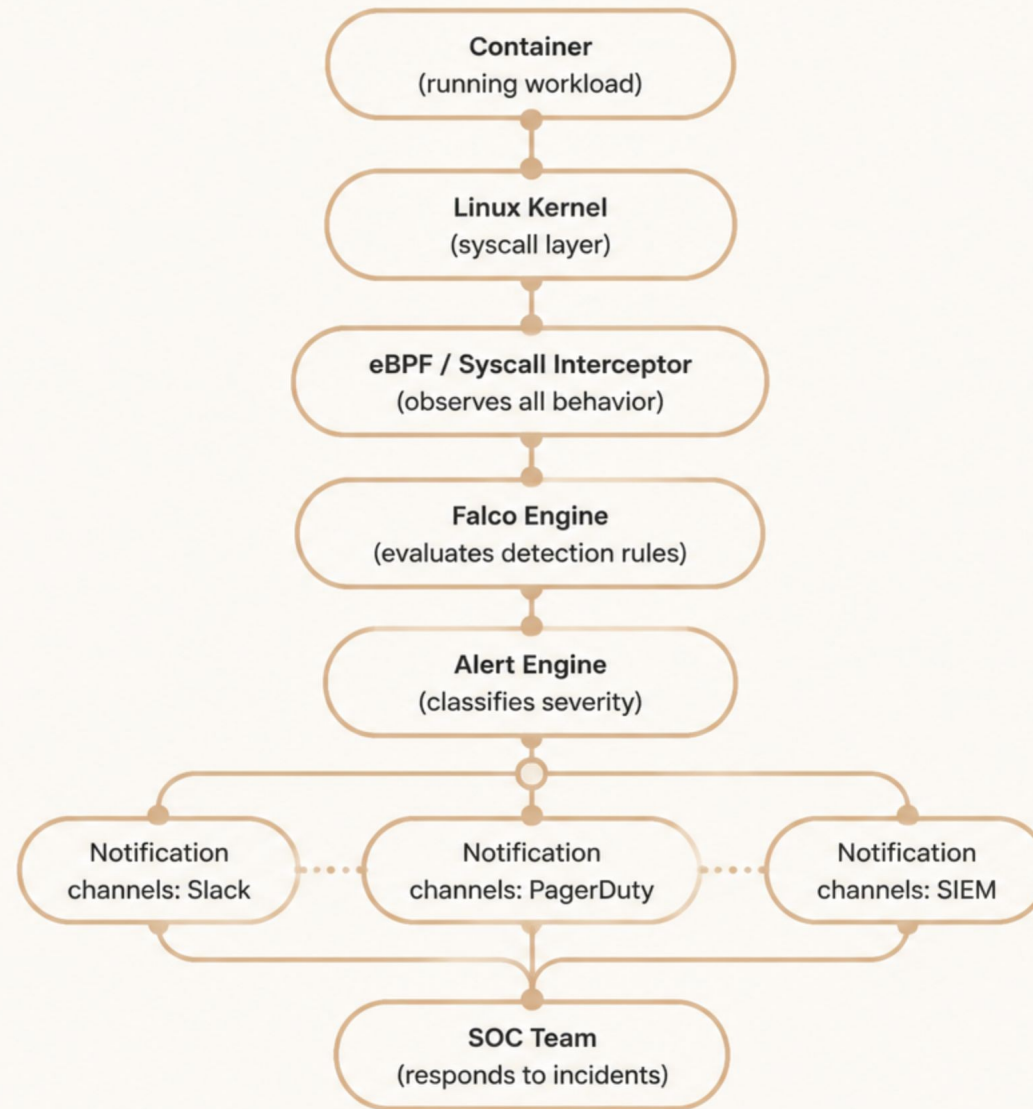
Lateral Movement & Data Theft

Compromised pods pivot through the network, steal secrets from environment variables or mounted volumes, and exfiltrate data via DNS tunneling.

Runtime protection tools like Falco and Tetragon detect these attack patterns in real time using eBPF-based syscall tracing — long before traditional alerting would fire.

Runtime Security Architecture

Runtime security leverages the Linux kernel's eBPF subsystem to intercept every syscall without modifying application code. Falco evaluates a rules engine in-kernel and routes alerts to downstream SIEM and SOC platforms.



This architecture shifts security from pure prevention to active detection and response — a critical capability for Zero Trust Kubernetes environments.

Runtime Security Tools

Open Source & Commercial Options for Production Environments

Falco

CNCF-graduated runtime detection engine. Uses kernel-level syscall tracing and a rich rules language to detect anomalous container behavior.

Tetragon

Cilium's eBPF-based security observability tool. Enforces policies directly in the kernel — preventing attacks before they complete, not just alerting after.

Cilium

Provides deep network visibility, identity-based policies, and transparent encryption for Kubernetes pod-to-pod traffic using eBPF.

Sysdig Secure

Commercial platform combining runtime protection, compliance scanning, forensics, and threat response in a unified enterprise UI.

AWS GuardDuty / Defender

Cloud-native threat detection from hyperscalers. Integrates with EKS and AKS for automated threat response workflows.

Policy as Code

Traditional Security Model

A human administrator manually reviews and approves every configuration change, deployment request, and infrastructure modification. This model is slow, error-prone, and fundamentally unscalable in Kubernetes environments running thousands of workloads.

Manual Approval Gates

Bottlenecks slow delivery and create inconsistent enforcement

Human Error

Reviewers miss edge cases; policies aren't uniformly applied

Policy as Code Model

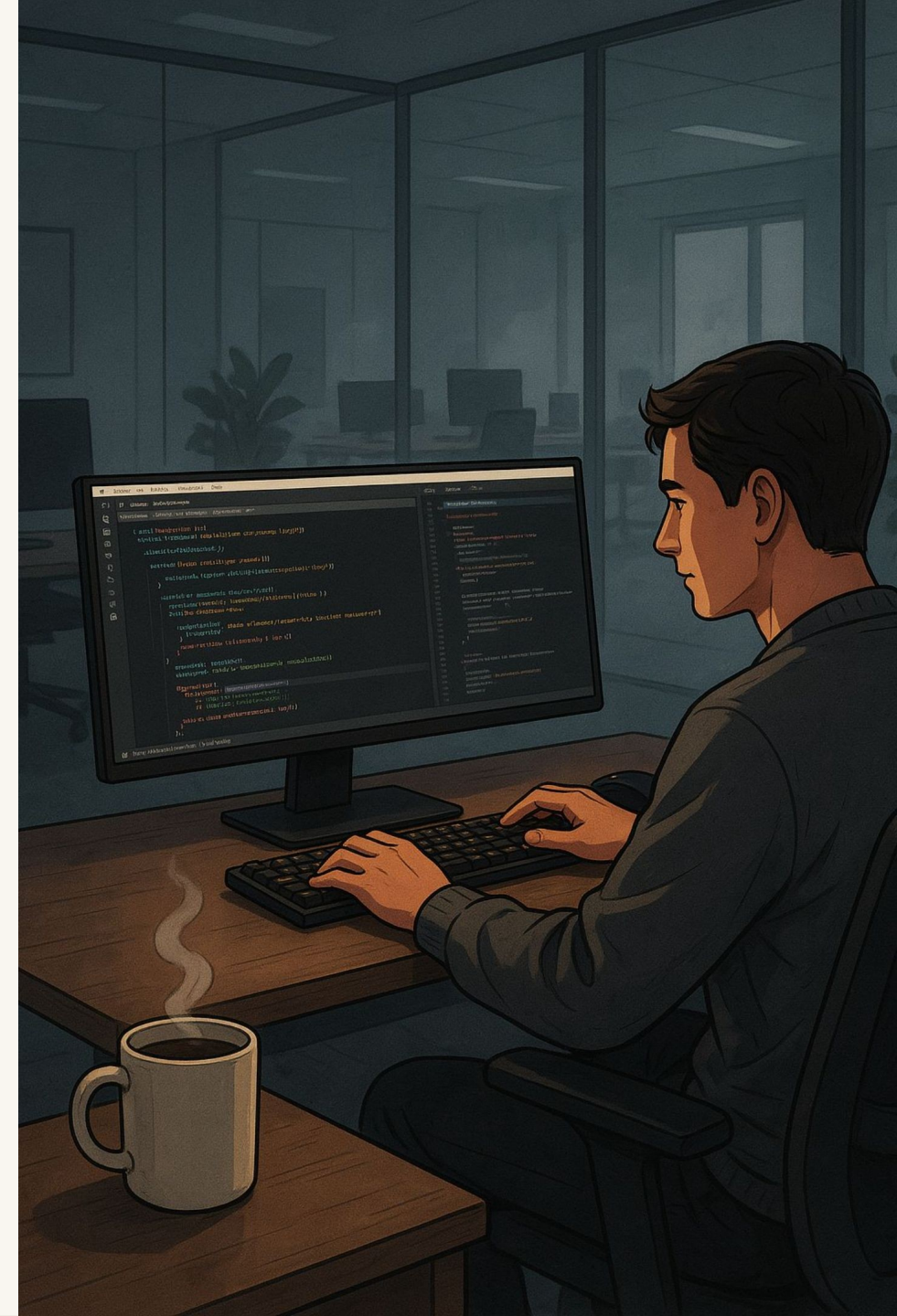
Policies are expressed as **executable code** checked into version control. The infrastructure automatically validates every deployment against these policies — no human in the loop required.

Automated Enforcement

Consistent, instant validation at cluster admission time

Version Controlled

Policy changes are auditable, reviewable, and reversible



Why Policy as Code?

From Human Judgment to Automated Guardrails

Instead of saying "developers should not run privileged containers" in a document nobody reads — the cluster automatically rejects them at admission time.



Automation

Every deployment is validated against policies without any manual review steps, dramatically accelerating delivery velocity.



Auditability

Policy definitions and enforcement decisions are stored in Git and logs — providing a complete, immutable audit trail for compliance teams.



Consistency

The same policy applies identically across every namespace, cluster, and environment — eliminating configuration drift.

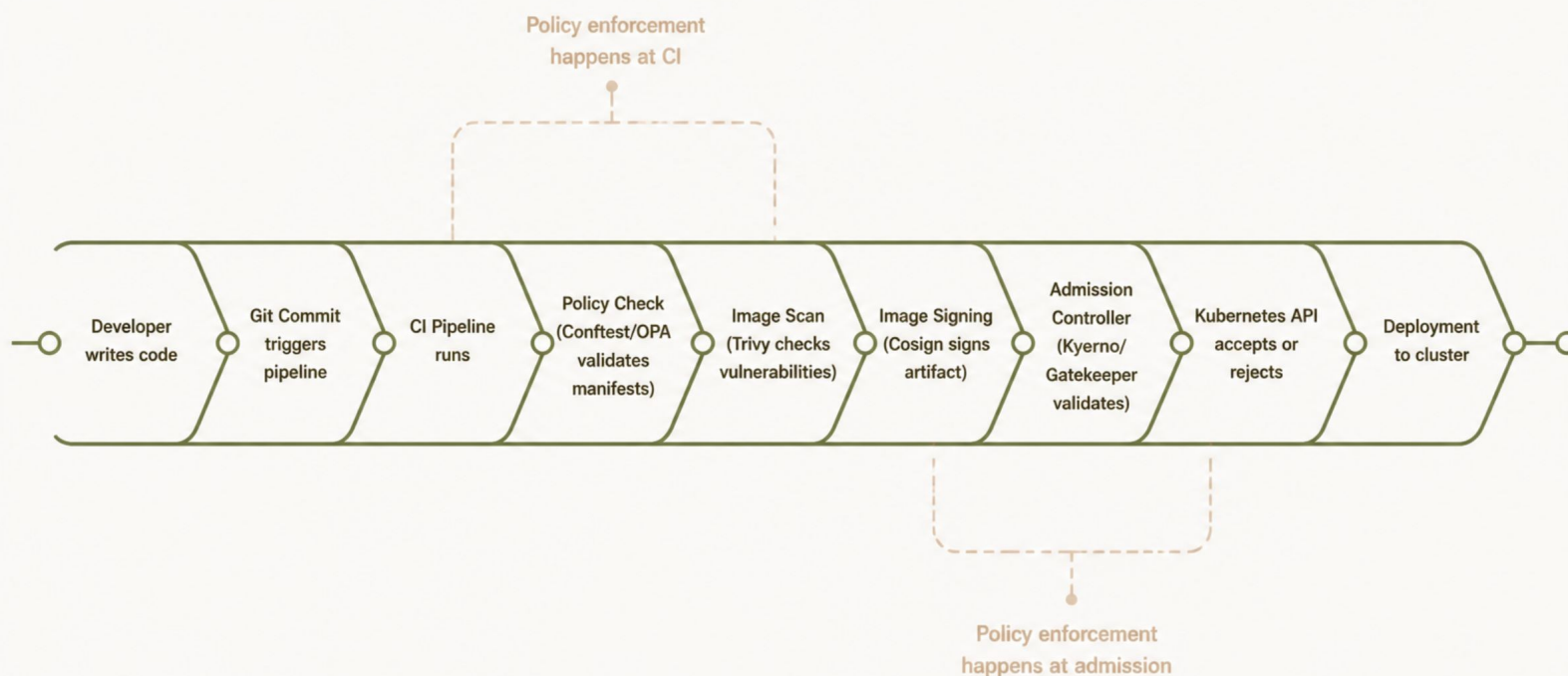


Compliance

Map policies directly to regulatory controls (SOC 2, PCI-DSS, NIST) and demonstrate enforcement programmatically during audits.

Policy Enforcement Pipeline

Policy as Code is enforced at **multiple stages** of the delivery pipeline — not just at cluster admission. This defense-in-depth approach catches misconfigurations as early as possible, reducing the cost of remediation.



📌 Security exists before production — catching policy violations in CI is 10× cheaper than catching them at runtime.

Policy Engines

Choosing the Right Tool for Your Kubernetes Environment

Kyverno

Kubernetes-native policy engine. Policies are written as Kubernetes resources (YAML) — no new language required. Supports validation, mutation, and generation.

OPA Gatekeeper

Uses the Rego policy language for highly expressive, general-purpose policies. Deep flexibility but requires learning Rego semantics and the constraint framework.

Kubewarden

Policy engine based on WebAssembly (WASM) modules. Policies can be written in Go, Rust, or any WASM-compatible language for maximum portability.

Conftest + OPA

Validates Kubernetes manifests, Helm charts, and Terraform configs in CI pipelines — before resources ever reach the cluster admission layer.

Real Enterprise Policies

What Policy as Code Looks Like in Production

Only signed images allowed

Kyverno verifies Cosign signatures on every image before admission

Images only from company registry

Blocks any image not originating from the approved internal OCI registry

No `latest` tag permitted

Enforces immutable, versioned image references to prevent ambiguous deployments

ReadOnlyRootFilesystem required

Prevents attackers from writing malicious files to the container filesystem at runtime

No privileged containers

Eliminates the most common container escape vector at the cluster level

Additional Mandatory Controls

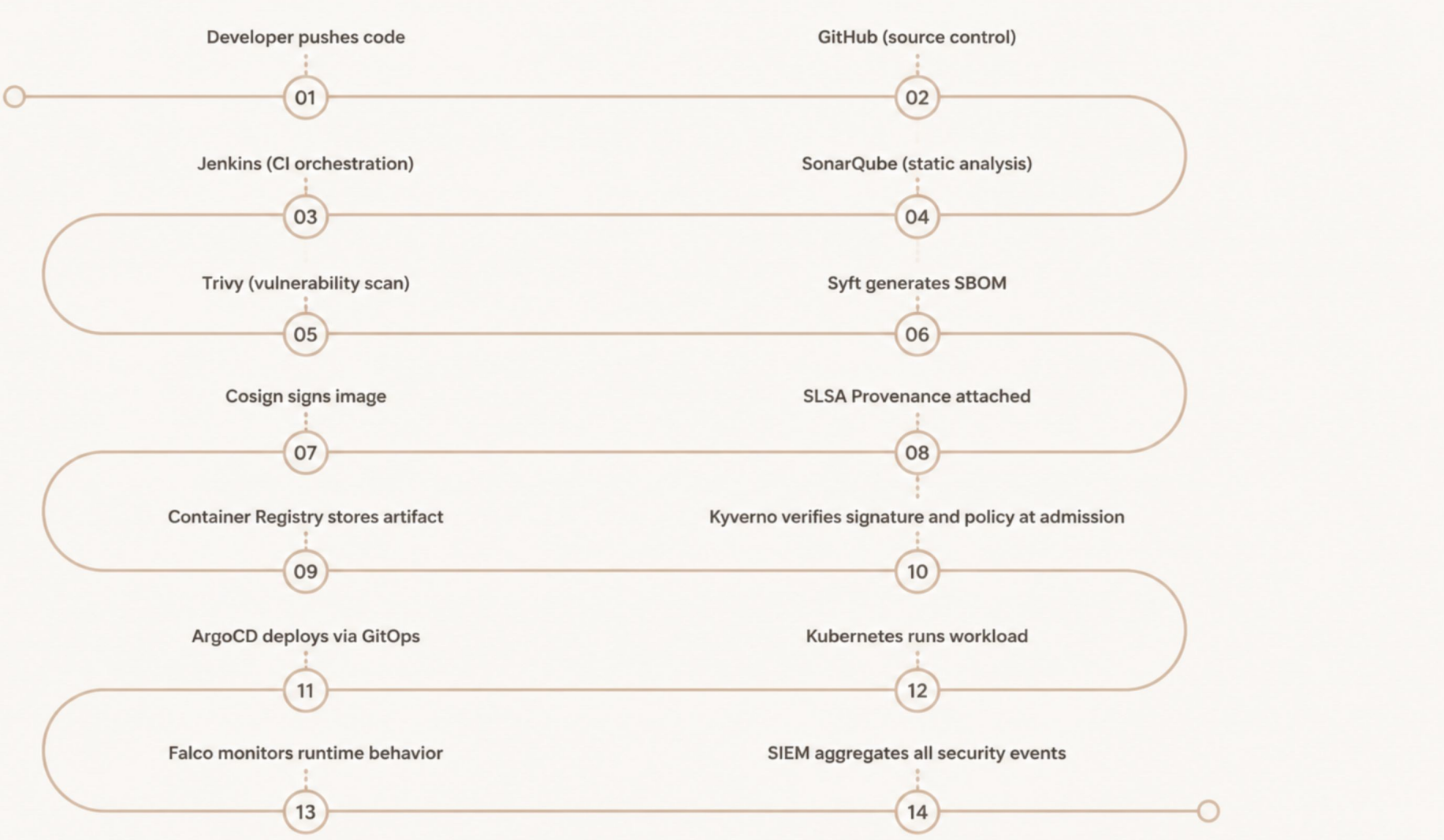
- Resource limits mandatory on all containers
- SecurityContext defined on every pod spec
- Non-root user required (`runAsNonRoot: true`)
- Approved namespaces only — no deployments to `default`

📋 These policies prevent the most common Kubernetes misconfigurations — automatically, at scale, every time.

End-to-End Secure Supply Chain

A Modern Enterprise DevSecOps Pipeline

This architecture integrates every security control — SBOM generation, signing, provenance, policy enforcement, and runtime detection — into a single continuous delivery workflow with no manual security gates.



Industry Best Practices

1

Adopt SLSA Level 3+

Target hardened build platforms with verified source integrity as your enterprise baseline.

2

Generate SBOM for Every Build

Use Syft in CI to produce SPDX or CycloneDX SBOMs and attach them as OCI attestations to every image.

3

Sign Every Artifact

Use Cosign with keyless signing (Sigstore) or KMS-backed keys. No unsigned artifact should ever reach production.

4

Enforce Policy as Code

Deploy Kyverno or OPA Gatekeeper in enforce mode across all clusters. Shift left with Conftest in CI pipelines.

5

Continuously Monitor Runtime

Run Falco or Tetragon in every production namespace. Route alerts to your SIEM and maintain active detection rules.

6

Automate Compliance

Map policies to regulatory controls and generate evidence automatically — never rely on manual audit preparation.

Key Takeaways

Five Questions — One Secure Supply Chain

SBOM

"What's inside?" A complete, machine-readable inventory of every dependency in your artifact.

Signing

"Who created it?" Cryptographic proof that the artifact was produced by a trusted, verified identity.

Provenance

"How was it built?" An immutable record of the build environment, inputs, and pipeline steps.

Policy as Code

"Should it be deployed?" Automated enforcement of security standards at every admission boundary.

Runtime Security

"Is it behaving safely?" Continuous syscall-level monitoring for anomalous activity post-deployment.



Together, these five controls create a complete, cryptographically verifiable secure software supply chain — from first commit to production runtime.

What's Next — Hands-on Labs

Apply What You've Learned in Real Kubernetes Environments

Lab Sequence

01

Generate SBOM with Syft

Produce a CycloneDX SBOM from a container image and inspect its dependency graph

02

Scan Vulnerabilities with Grype

Run Grype against your SBOM and identify critical CVEs before promotion

03

Sign Images with Cosign

Attach a keyless Cosign signature and verify it against the Rekor transparency log

04

Generate SLSA Provenance

Use the SLSA GitHub Generator to produce and attach L3 provenance attestations

Advanced Labs

01

Enforce with Kyverno

Write and deploy image verification policies; test enforcement by attempting to deploy unsigned images

02

Runtime Detection with Falco

Simulate a reverse shell and privilege escalation; observe Falco alerts in real time

03

Full Pipeline Integration

Build the complete Jenkins → Trivy → Syft → Cosign → ArgoCD → Kubernetes → Falco pipeline end-to-end



Module 11 Complete

From Source Code to Runtime — Fully Secured.

You've completed the full supply chain security lifecycle: provenance, SBOM, signing, policy enforcement, and runtime detection. These aren't isolated tools — they form a layered, defense-in-depth architecture that protects every stage of software delivery in a modern Kubernetes enterprise environment.

Review

Revisit slides on SLSA levels, Falco architecture, and Kyverno policies



Practice

Complete all 7 hands-on labs before the next module

Next Module

Module 12: Advanced Kubernetes Threat Modeling & Incident Response